Safest Route on Mars

Every parameter that matters, and the algorithms to turn them into a route

For the MarsWalk / ELYSIUM route engine (Sanket Grid + weighted A*)

What this document gives you

- **Section 1** - the core idea: define 'safe' as survival probability, so every hazard becomes one additive cost.
- **Section 2** - the full parameter catalogue (about 90 factors in 11 groups) with the reason it matters, the dataset it comes from, and a starting safe-to-limit range.
- **Section 3** - the formulas that convert raw layers (slope, wind, pressure, dust, radiation, battery, oxygen) into per-cell numbers.
- **Section 4** - the route algorithm: hard constraints, the risk cost function, weighted A*, shelter-reachability pruning, resource budgets, uncertainty, rerouting.
- **Section 5** - route modes (safest, fastest, balanced...) as weight presets, plus the route score and custom-route audit.
- **Section 6** - what to add to your Sanket Grid table, a build order, and how to validate.
- **Appendix** - a working Python reference implementation (tested on a synthetic grid).

Read this first. The numeric thresholds and weights here are sensible **starting values** taken from general rover / EVA practice and physics, not mission-certified limits. Replace them with the specs of your actual vehicle or suit, and calibrate against real traverse data (Section 6). The structure of the method is what matters; the numbers are meant to be tuned.

1. The core idea: safety as survival probability

A route is a chain of small steps (edges) between grid cells. Each hazard has some chance of ending the mission when you are exposed to it. If the hazards are treated as independent, the chance of getting through the whole route is a product:

```
P(survive route) = product over all edges e of  (1 - p_e)  ~  exp( - sum_e r_e )

r_e = -ln(1 - p_e)  ~  p_e       "risk" of edge e (small, additive, >= 0)
Route risk   R = sum_e r_e
Safety score = 100 * exp(-R)     (0..100 %)
```

Why this matters for your A* search: taking the logarithm turns a multiplication into a sum, and A* (and Dijkstra) minimise sums. So minimising total risk *is* maximising survival probability, and every different hazard (slope, dust, radiation, cold, comms loss...) just adds one more term to r_e. That gives you one consistent number instead of arguing over how to compare 'a bit of slope' with 'a bit of radiation'.

Three kinds of factors, handled differently:

Type	What it is	How it enters the algorithm
Hard constraint	Physically impossible or unacceptable: slope above vehicle limit, cliff, deep sand, wheel slip too high, boulder taller than clearance.	Cell / edge is removed from the graph. A* can never use it. No weighting - you do not trade a cliff against a shorter route.
Soft risk (per metre)	Hazard you are exposed to by distance travelled: moderate slope, roughness, rocks, dunes, nearness to cliffs.	r = distance x lambda x h(x), where h(x) is a 0..1 ramp from 'safe' to 'limit'.
Soft risk (per hour)	Hazard you are exposed to by time spent: dust opacity, wind, cold, radiation dose, comms loss, solar-storm exposure.	r = time on edge x mu x h(x), plus a dose term k_rad x mSv.
Budget (resource)	Finite supplies: oxygen, battery, EVA hours, radiation dose limit, water.	Constrained search: Lagrange multipliers, or 'can I still get back?' pruning (Section 4.5-4.6).

The ramp function (used for every soft factor)

```
ramp(x; a, b) = clamp( (x - a) / (b - a), 0, 1 )      a = 'safe' value, b = 'limit' value
h(x) = ramp(x; a, b) ^ q                                q = 2 (so mild values are cheap, near-limit values are
     expensive)
```

Example: slope ramp from 8 deg (safe) to 15 deg (limit): 8 deg costs 0, 11.5 deg costs 0.25 of the maximum, 14 deg costs 0.73, and anything above 15 deg is a hard block.

2. Parameter catalogue

Columns: **Why it matters** for route safety | **Source** (dataset from your ELYSIUM list where one exists) | **Safe -> Limit** = starting values for the ramp (HARD = remove from graph). Vehicle-dependent numbers are marked (V). Everything is configurable.

A. Terrain geometry

Parameter	Why it matters	Source / how to get it	Safe -> Limit (start values)
Elevation z	Base for slope; proxy for pressure, temperature and radiation shielding; climb energy.	MOLA/HRSC DEM, HiRISE DTM	Input only
Along-track slope	Wheel slip, braking, energy, falling for a walker.	DEM, computed per edge	8 deg -> 15 deg HARD (V)
Cross-slope (side slope)	Rollover / sliding sideways; the top rover-loss mode on traverses.	DEM + heading	8 deg -> 18 deg HARD (V)
Slope at several baselines	A 5 m slope hides 1 m steps. Compute at 1, 5, 20 m and use the max.	HiRISE DTM 1 m	Same thresholds
Aspect	Sun exposure, wind exposure, downslope direction (slip and rockfall direction).	DEM	Input only
Profile / plan curvature	Convex crests = exposed edges; concave = dust and cold pooling, traps.	DEM	Flag |curv| above 0.1 /m
Roughness (RMS height)	Shock loads, getting stuck, speed loss, sensor jitter. Use 1 m, 5 m, 25 m scales.	MOLA roughness (coarse); HiRISE-derived (fine)	0.1 -> 0.3 m RMS
Step height / local relief	Single ledges and ridges beyond wheel or boot capability.	HiRISE DTM, 3 x 3 window max-min	0.15 -> 0.30 m HARD (V)
Distance to cliff / scarp	Edge falls; unstable ground near edges; rockfall runout below cliffs.	Slope above 35 deg, distance transform	Risk decays with e^(-d/30 m); HARD within 5 m
Crater rim / wall geometry	Steep, unstable walls; crater floors can trap.	Robbins crater DB	Treat wall slope as above
Rockfall runout zone	Boulders roll downslope from steep source cells.	Slope > 30 deg sources + downslope buffer	Flag zone, risk 0 -> 1 with distance
Sky-view factor / horizon	Controls shadow, radiation shielding, solar power, comms visibility.	Viewshed from DEM	Input to other terms
Corridor width / lateral clearance	Room to dodge a bad patch; narrow safe corridors are fragile.	Distance transform of passable cells	Risk decays with e^(-d/30 m)

B. Surface material and trafficability

Parameter	Why it matters	Source / how to get it	Safe -> Limit (start values)
Rock abundance (CFA %)	Boulder fields block wheels and feet; hidden sharp rocks damage wheels.	MGS TES rock map; HiRISE rock counts	5 % -> 20 %
Individual boulder size	Anything taller than clearance is impassable.	HiRISE boulder detection	Height > 0.3 m HARD (V)
Dunes / ripples / TARs	Deep loose sand: sinkage, slip, entrapment. Opportunity rover and Spirit were both lost or crippled in such terrain.	MGD3 dune DB, AI4Mars 'sand'	Sand polygons: HARD for rovers, high penalty on foot
Terrain class	Soil / sand / bedrock / big rock gives baseline slip and sinkage.	AI4Mars labels	Table of s0, phi per class
Base slip s0 and friction angle phi	Feeds the slip model (Section 3.3).	Class table, calibrated	s0: bedrock 0.05, soil 0.10, sand 0.35; phi 20-35 deg
Bearing / cohesion (Bekker kc, kphi, n)	Sinkage and rolling resistance.	Class table, rover testbeds	Sinkage > 0.3 x wheel radius = HARD (V)
Thermal inertia	Low TI = fine dust or loose sand (soft); high TI = rock or bedrock.	TES / THEMIS	Low TI (< 150 SI) -> raise slip
Albedo	Bright = deep dust cover (a trap and a visibility issue).	MGS TES albedo	Flag very bright

Parameter	Why it matters	Source / how to get it	Safe -> Limit (start values)
Slope streaks / RSL	Signs of unstable, flowing granular slopes.	HiRISE catalog	HARD within buffer
Subsurface voids / lava tubes	Roof collapse under weight; also a radiation shelter (opportunity).	MGC3 cave catalog, gravity anomalies	HARD above void roof if thin
Ice-cemented / brittle crust	Crust that breaks through unpredictably.	SWIM, SHARAD (indirect)	Raise uncertainty term
Dust-devil tracks	Indicate active wind-lofted dust zones.	HiRISE / CTX tracks	Mild penalty

C. Atmosphere: wind, pressure, dust, visibility

Note on your requested wind and pressure inputs: Mars surface pressure is only about 600 Pa (roughly 0.6 % of Earth), so a 30 m/s wind pushes with a dynamic pressure of only about 9 Pa, far too weak to knock over a rover or an astronaut. Wind and pressure matter **indirectly**: they raise dust (visibility, solar panel loss, abrasion), set convective cooling, and signal storms. Pressure is mainly an input to density.

Parameter	Why it matters	Source / how to get it	Safe -> Limit (start values)
Wind speed (mean and gust)	Dust lifting, saltation, sandblasting, dust-devil encounters, cooling of suit and rover.	Mars Climate DB / MCS profiles, MarsWRF; REMS-type station data	15 -> 30 m/s gust
Wind direction	Dust-plume drift; which side of a ridge is sheltered; downwind dune migration.	As above	Input
Surface pressure P	Sets air density and convective heat loss; varies with season (about 20 %) and with elevation.	MCD, hypsometric formula from DEM	Input to density
Air density rho	Wind load, heat transfer, helicopter or drone use.	rho = P / (R T)	Input
Dust opacity tau	Cuts solar power exponentially, reduces visibility, signals storm. tau above 3 is a severe regional storm.	MCS, MARCI dust storm DB (MDSSD)	1.0 -> 4.0
Dust storm season / Ls	Storm risk peaks roughly Ls 180-360; global storms are rare but last weeks.	MARCI storm sequences, SPICE for Ls	Raise mu_dust in season
Dust-devil frequency	Brief wind spikes and short dust events; they can also clean panels.	HiRISE / MCS	Mild
Visibility range	Navigation by sight, hazard detection distance.	From tau	Under 200 m: reduce speed / stop
Frost / CO2 ice (high latitude)	Slippery ground, hidden crevasses on caps.	Climate DB by Ls and latitude	HARD at cap edges
Static charging of dust	Dust clinging to suit seals and joints (minor).	-	Ignore in v1

D. Thermal environment

Parameter	Why it matters	Source / how to get it	Safe -> Limit (start values)
Air / surface temperature T(t)	Mars swings from about +20 C to -120 C or lower; cold drives heater power and suit load; extreme cold stresses materials.	MCD, THEMIS, MCS	T below -80 C -> -120 C
Local solar time / sunset	Daylight window sets the allowed EVA length; night is the cold-soak and power hazard.	SPICE ephemeris	Must finish with margin before sunset
Shadow and sun exposure	Sun on slope aspect changes surface temperature and solar charge.	DEM + sun vector	Input
Heater load	Power drained to survive the cold; adds to energy budget.	Thermal model	Input to energy
Thermal gradient on suit	Cold ground contact vs sunlit side; local suit limits.	Suit spec	Spec-based

E. Radiation

Parameter	Why it matters	Source / how to get it	Safe -> Limit (start values)
GCR dose rate	Chronic exposure. Order of 0.2-0.7 mSv per day equivalent at the surface (Curiosity RAD at Gale measured roughly 0.6-0.7); varies with solar cycle.	MSL RAD data, atmospheric depth model	Input to dose accumulation
Atmospheric column depth	More overhead atmosphere = a bit more shielding, so low basins (like Jezero) are better than high plateaus.	DEM + pressure model	Fit dose vs elevation to data
Terrain shielding (sky-view factor)	Cliffs and crater walls block part of the sky; caves block almost all of it.	Viewshed from DEM	Dose x (0.5 + 0.5 x SVF)
Solar particle event (SPE) risk	Rare but acute; can deliver a dangerous dose within hours. Route must stay within reach of shelter.	MAVEN SEP logs, space-weather alerts	Shelter within warning time
SPE warning lead time	How long you have to reach shelter once an alert is issued.	Operations assumption (about 1 h)	Configurable
Shelter locations	Rover cabin, hab, lava tubes, deep crater overhangs, regolith berms.	MGC3 cave catalog, mission plan	Required for reachability
Time to nearest shelter	Computed per cell, drives the SPE and 'can I get home' pruning.	Backward Dijkstra (Section 4.5)	Under warning time
Cumulative dose / EVA limit	Per-EVA and per-mission dose caps (organisation sets the career limit).	Mission rules	Budget (Section 4.6)

F. Power and energy

Parameter	Why it matters	Source / how to get it	Safe -> Limit (start values)
Solar irradiance	About 590 W/m^2 above the atmosphere on average, varying with the Sun distance; attenuated by tau and sun angle.	Ephemeris, tau	Input to power
Sun incidence angle	Panel tilt vs slope aspect and time of day.	DEM + SPICE	Input
Battery state of charge	Cannot cross a section you cannot finish; low SOC adds shutdown risk.	Vehicle telemetry	Keep 20-25 % reserve (V)
Climb energy	m g dh over uphill sections, plus rolling resistance.	DEM	Budget
Regeneration downhill	Some energy returns on descents.	Vehicle spec	Efficiency 0.3-0.6 (V)
Heater / night survival drain	Keeping electronics warm uses battery overnight.	Thermal model	Budget
Rolling resistance Crr	Rises sharply with sinkage in soft soil.	From terrain class	Per-class table

G. Life support and human factors

Parameter	Why it matters	Source / how to get it	Safe -> Limit (start values)
Metabolic rate along route	Walking uphill or in heavy suit burns more; sets O2 and cooling demand. The Minetti cost-of-walking curve gives energy vs gradient (Earth-calibrated).	Gradient per edge	Budget
Oxygen consumption	About 0.07-0.10 kg per hour at EVA exertion (4 MET example).	Metabolic model	Reserve >= return trip + 30 %
CO2 scrubber, coolant, power of suit	Consumables with their own limits.	Suit spec	Budget
EVA duration	Fatigue, thermal and consumables all grow with time.	Mission rules	e.g. 8 h max
Walk-back distance	Distance you can cover on remaining O2 plus a margin; the farthest point you are allowed from refuge.	Derived	Hard prune (Section 4.5)

Parameter	Why it matters	Source / how to get it	Safe -> Limit (start values)
Fatigue / rest stops	Long monotone effort raises mistake risk; spaced rest points add safety.	Operations	Rest every 1-2 h
Pace, load carried	Slower on slopes and with equipment.	Operations	Config
Medical / injury state	A reduced-capability walker needs gentler limits.	Crew state	Scale all limits

H. Communications and navigation

Parameter	Why it matters	Source / how to get it	Safe -> Limit (start values)
Line of sight to hab / relay	Loss of voice and telemetry; no help possible. Computed by viewshed from the DEM.	DEM viewshed	Comms block flag 0/1
Orbiter pass windows	Windows when relay is overhead for UHF / X-band.	SPICE kernels	Schedule check
Localisation quality	Featureless plains give poor visual landmarks and drift; also under dust.	Texture / HiRISE feature density	Penalty on sparse areas
Map and hazard-detection range	How far ahead the rover or suit sensors can see a hazard.	Sensor spec, tau	Input to speed
Position uncertainty	Drift widens the true corridor, so require extra lateral clearance.	Navigation filter	Add to clearance

I. Geological and event hazards

Parameter	Why it matters	Source / how to get it	Safe -> Limit (start values)
Marsquake / fault proximity	Ground failure, rockfall triggering, dust lofting.	InSight SEIS catalog	Mild buffer near faults
Gravity anomaly	Hints at voids or density contrasts.	MRO / MGS gravity	Uncertainty input
Water ice / brine ground	Ice-cemented layers; possible weak ground; also science and resource value.	SWIM, SHARAD, Odyssey GRS	Flag
Perchlorate / toxic dust	Health hazard on suit entry; a consideration for dust-intensive zones.	CRISM mineralogy	Operational flag
Polar cap crevasses / CO2 frost	Only relevant at high latitude.	Climate DB, imagery	HARD

J. Operations, timing and contingency

Parameter	Why it matters	Source / how to get it	Safe -> Limit (start values)
Season Ls and local sol time	Combine with dust, temperature, daylight.	SPICE	Time-dependent fields
Safe-haven spacing	Distance to nearest shelter or cache at every point.	Backward Dijkstra	Under walk-back distance
Route redundancy / alternates	A second safe route if the first closes.	k alternative routes	At least 1 alternate
Route complexity	Number of turns, direction changes; easier to follow and verify.	Path metric	Minor penalty
Vehicle / suit class	Selects all (V) limits above; a rover and a walker differ.	Config profile	Profile file
Mission margin policy	Required slack in time, O2 and battery.	Operations	20-30 % margin
Science value (optional)	Not a safety factor; adds a reward in 'Optimum' route only (your Science Explorer layer).	Your science targets	Weighted in Section 5

K. Data quality (often forgotten, but it controls how safe the answer really is)

Parameter	Why it matters	Source / how to get it	Safe -> Limit (start values)
DEM resolution and error sigma_z	A 200 m DEM cannot see a 1 m ledge. Use HiRISE where available; otherwise add margin.	Dataset metadata	Use x_eff = x + z sigma
Co-registration error	HiRISE vs CTX vs DEM misalignment shifts hazards by several metres.	Registration check	Inflate buffers
Label confidence	AI4Mars-style labels and classifiers are probabilistic.	Model confidence	Use probability-weighted risk
Image age / seasonal change	Dunes migrate and surfaces change.	Acquisition date	Penalise old data
Coverage gaps	No data = unknown, not safe.	Mask	Treat unknown as high cost or blocked

3. Formulas that turn raw layers into per-cell and per-edge numbers

Notation: (i, j) = row, column of the Sanket Grid; d = edge length in metres; heading unit vector u = (ux, uy) with x along columns and y along rows.

3.1 Slope, aspect and cross-slope

```
Horn gradients on DEM:   gx = dz/dx ,  gy = dz/dy        (finite differences over a 3x3 window)
Slope angle:             theta = atan( sqrt(gx^2 + gy^2) )
Aspect (downslope dir.): alpha = atan2(-gx, -gy)
Along-track (edge a->b): tan(theta_al) = (z_b - z_a) / d         (signed: + uphill)
Cross-slope:             tan(theta_cr) = | gx*uy - gy*ux |
Multi-scale slope:       theta_eff = max( theta_1m , theta_5m , theta_20m )
```

3.2 Roughness and step height

```
Roughness (RMS) over a window of n cells after removing the local plane:
    rough = sqrt( (1/n) * sum( (z_k - plane_k)^2 ) )
Step height = max(z) - min(z) over the cells the wheel / boot spans (3x3 window)
```

3.3 Wheel slip and speed (engineering approximation - calibrate)

```
slip s = s0 + (1 - s0) * min(1, max(tan(theta_al),0) / tan(phi))^2
         s0 = base slip of terrain class, phi = friction angle of the soil

speed v = v0 * (1 - s) * min( 1, exp( -3.5 * ( |tan(theta_al) + 0.05| - 0.05 ) ) )     (Tobler-style slope
    factor)

Hard block:  s > s_max (0.6)  OR  |theta_al| > theta_max  OR  theta_cr > theta_cross_max
Better still: replace s() by an empirical slip-vs-slope curve fitted from real rover drive logs for each
    AI4Mars class.
```

3.4 Atmosphere: density, wind load, solar power

```
Density          rho = P / (R * T)               R = 188.9 J/(kg K)  (CO2 atmosphere)
                 e.g. P = 610 Pa, T = 210 K  ->  rho ~ 0.0154 kg/m^3
Pressure vs z    P(z) = P_ref * exp( -(z - z_ref) / H )     H ~ 10.8 km scale height
Wind dyn. press. q = 0.5 * rho * v^2             (30 m/s -> about 7-9 Pa: negligible mechanically)
Dust visibility  extinction k ~ tau / H_dust ;  visibility ~ 3.9 / k       (use for speed limits)
Solar power      P_sol = A * eta * S0 * (1.524/r_AU)^2 * cos(i) * exp( -tau / cos(z_sun) )
                 S0 ~ 590 W/m^2 at mean Mars-Sun distance, i = panel incidence, z_sun = sun zenith angle
Sol daylight     use sunrise / sunset from SPICE; EVA end time <= sunset - margin
```

3.5 Thermal load

```
Heater power     P_heat = max( 0 , (T_set - T_amb) / R_th - P_internal )
Cold risk        h_cold = ramp( 200 K - T_amb ; 0 , 40 )         (starts at 200 K, saturates at 160 K)
Night survival   SOC_dawn = SOC_dusk - (P_heat + P_base) * t_night / E_battery  >= SOC_min
```

3.6 Radiation

```
Open-sky GCR dose rate     D0(z, t)  [mSv/day]    from measurement or atmospheric depth model; lower at low z
Terrain shielding          D_local = D0 * (0.5 + 0.5 * SVF)     SVF = sky-view factor, 0..1
                           in a cave or tube:  SVF ~ 0  ->  D_local ~ 0.5 * D0 or less (add roof mass term)
Dose on edge               dose_e = D_local / 24 * t_e                    [mSv], t_e in hours
SPE exposure model         T_exposed = max( 0 , T_shelter(cell) - T_warning )
                           dose_SPE = D_spe * T_exposed * (0.5 + 0.5*SVF)
                           risk term mu_spe * ramp( T_shelter ; T_warning , 2*T_warning )
Route dose                 Dose = sum dose_e   must stay <= EVA dose limit (Section 4.6)
```

3.7 Energy and oxygen

```
Rover energy (edge)   E = [ m*g*max(dz,0) + m*g*Crr*d ] / eta_drive  -  eta_regen * m*g*max(-dz,0)  +  P_aux *
    t
                      g = 3.71 m/s^2 ; Crr rises with sinkage
Walker metabolic power  M(i) per Minetti polynomial of gradient i (fraction):
                      C(i) = 280.5 i^5 - 58.7 i^4 - 76.8 i^3 + 51.9 i^2 + 19.6 i + 2.5   [J/(kg m)]   Earth-
                          calibrated;
                      apply a Mars-gravity correction to the mechanical part, then add suit and load mass.
Oxygen (edge)         m_O2 = ( MET * 3.5 mL/(kg min) * body_mass * 60 * t_e / 1000 ) * 1.429 g/L / 1000   [kg]
                      (4 MET and 80 kg -> about 0.096 kg/h)
Reserve rule          O2_remaining >= O2_to_nearest_shelter * 1.3  at every point along the route
```

3.8 Communications and line of sight

```
comm_block(cell) = 1 if no line of sight from the cell to hab / relay antenna, else 0
Line of sight test: sample the DEM along the ray; blocked if z_dem > ray_height + safety margin.
Orbiter relay:      pass windows from SPICE ephemeris kernels; schedule check, not a spatial cost.
```

3.9 Uncertainty (risk-averse inputs)

```
Use a pessimistic value of each uncertain input:   x_eff = x + z * sigma_x       z = 1.28 (90 %) or 1.64 (95
    %)
For classified terrain use the expected risk:      r = sum_classes  P(class) * r(class)
Unknown / no-data cells:  block, or give them the highest finite cost.
```

4. The route algorithm

The recommended method is **weighted A* on the Sanket Grid with a survival-probability cost**, then optional constraint handling for oxygen / battery / dose, and a re-plan loop when conditions change.

4.1 Pipeline

```
raw layers (DEM, rocks, dunes, tau, wind, temp, radiation, comms)
      |  normalise with ramp()  ->  per-cell hazard fields h_k  and  masks (blocked / comm_block)
      v
edge function:  hard-constraint test  ->  risk r_e, time t_e, dose, O2, energy
      v
shelter field:  backward Dijkstra from shelters  ->  T_shelter(cell)   (SPE + 'can I get home' pruning)
      v
A* (8-neighbour, no corner cutting)  with cost = risk + weights*(time, energy, ...) + multipliers*(resources)
      v
post-process: smooth path with line-of-sight test using the SAME edge function
      v
route statistics, safety score, alternatives, audit report  ->  Mission Dashboard / SOALM / 3D
```

4.2 Hard constraints (edge is removed)

- Along-track slope above the vehicle limit, or cross-slope above the side-slope limit.
- Wheel slip above s_max (e.g. 0.6), sinkage above the limit, or any step / boulder taller than clearance.
- Cliff edge, slope streak or void buffer, dune polygons (for rovers), polar crevasse zones.
- No-data cells, and cells with high-severity hazard value above the 'High' band (> 0.75) in your hazard system.
- Diagonal moves that cut a blocked corner (check both orthogonal neighbours).
- Cells beyond the walk-back radius (Section 4.5).

4.3 Edge risk (the heart of the method)

```
Per-metre terms  (multiplied by distance d):
    r_dist = d * [  lam_slope * h(theta_al)  + lam_cross * h(theta_cr) + lam_slip * h(slip)
                  + lam_rough * h(rough) + lam_rock * h(rocks) + lam_dune * h(dune)
                  + lam_clear * exp(-clearance / d0) ]

Per-hour terms   (multiplied by edge time t_e):
    r_time = t_e * [ mu_dust * h(tau) + mu_wind * h(gust) + mu_cold * h(cold)
                   + mu_comm * comm_block + mu_spe * ramp(T_shelter ; T_w , 2*T_w) ]

Radiation term:     r_rad = k_rad * dose_e                          (linear, no safe threshold)

Edge risk:          r_e = r_dist + r_time + r_rad
Edge cost for A*:   c_e = r_e + eps * d + beta * t_e       eps tiny, keeps routes short; beta = time pressure
```

Using an **edge integral** (distance or time) rather than adding the cell value once per cell matters: a diagonal step is 1.41 times longer than a straight one and must cost 1.41 times more exposure, or paths will zig-zag. Average the cell values at both ends of the edge, as the reference code does.

4.4 Weighted A* search

```
Open set = priority queue ordered by  f(n) = g(n) + h(n)
g(n)     = cheapest known cost from start to n
h(n)     = c_min * octile_distance(n, goal) * cell_size            (admissible heuristic)
           c_min = smallest possible cost per metre = eps + beta / (v0 * 3600)
Expand n: for each of the 8 neighbours m (skip blocked, skip corner cuts):
            e = edge(n, m);  if e is forbidden: continue
            if g(n) + c_e < g(m):  update g, parent, push
Stop when goal is popped.  Optimal for the cost function as defined.

Notes
 - Where terrain is uniformly safe, risk per metre is ~0, so A* behaves like Dijkstra. That is fine up to
     about 10^6 cells.
 - For larger maps use tiles / hierarchical A*, or run on a coarser grid then refine near the route.
 - Grid paths have 45-degree bias: smooth with string-pulling (shorten if line-of-sight passes the edge test),
     or use Theta*.
 - Use 8-neighbour moves; 4-neighbour gives long, rigid paths.
```

4.5 'Never go where you cannot get back' (shelter-reachability pruning)

```
1) Backward Dijkstra from every shelter / hab using the same edge function, with edge weight = travel time.
     T_shelter(cell) = hours from the cell back to the nearest shelter.
2) SPE term: mu_spe * ramp( T_shelter ; T_warning , 2 T_warning )  adds risk where shelter is farther than the
    warning time.
3) Hard prune in forward A*:
```

```
      reject node n if   t_used(n) + margin * T_shelter(n)  >  t_budget          (margin ~ 1.3)
      reject node n if   O2_used(n) + margin * O2_return(n) >  O2_budget
The same field also gives the immediate 'run home' route when an SPE warning arrives.
```

4.6 Resource budgets (oxygen, battery, time, dose)

The cheapest-risk path might burn too much oxygen or battery. Handle this as a **constrained shortest path**:

```
Minimise   Risk(path)
subject to Time(path) <= T_max ,  O2(path) <= O2_max ,  Dose(path) <= D_max ,  Energy(path) <= E_max

Lagrangian relaxation (implemented in the appendix code):
    cost_e(lambda) = r_e + lambda_t * t_e + lambda_o * o2_e + lambda_d * dose_e
    loop: run A*; for each violated limit, increase its multiplier (x2); stop when all limits are met.
Exact alternative: state-augmented A* with label (cell, battery bucket, O2 bucket); larger but exact.
```

4.7 Time-dependent conditions

- Fields that change with time (temperature, sun angle, tau, wind) are evaluated at the **arrival time** at the cell: t = t_start + travel time so far. Because waiting never helps in this model (FIFO property), A* stays valid.
- Add a hard 'be in shelter or the hab by sunset minus margin' constraint using the shelter field.
- Forecasts of dust storms or SPE alerts are loaded as time-stamped masks; the planner re-runs when they update.

4.8 Safety variations you can switch on

Variant	Cost / rule	When to use
Sum of risks (default)	Minimise sum r_e = maximise survival probability	Normal planning
Bottleneck (minimax)	Minimise the single worst edge: binary search on a risk threshold T, keep edges with r_e <= T, test connectivity	When one bad step is unacceptable (e.g. crew on foot)
Risk-averse (CVaR-style)	Run on x_eff = x + z sigma for uncertain inputs, or Monte Carlo and take the 90th-percentile route risk	Low-quality data
Widest corridor	Maximise minimum lateral clearance; use distance transform as an extra term	Poor localisation or dust
k alternatives	Penalise cells of route 1 by x1.5 (or use Yen / penalty method), re-run	Backup routes

4.9 Dynamic rerouting triggers

- New hazard seen by sensors, or a blocked cell: update that cell and re-plan (D* Lite / LPA* update only the affected part).
- tau or gust above threshold, or temperature below limit: raise mu terms and re-plan, or abort to shelter.
- SPE alert: immediately follow the backward-Dijkstra field to the nearest shelter.
- O2, battery or time below reserve for the remaining route: re-plan toward the nearest refuge, then return.
- Log: old route, new route, distance and time difference, reason - this is exactly the data your Section 9 (Dynamic Rerouting) list needs.

5. Route modes, scoring and custom-route audit

Different route modes are the **same engine** with different weights. Weights multiply the lambda / mu terms and the non-safety objectives, each divided by a reference value so units are comparable (e.g. T_ref = straight-line time).

```
C_e = w_s * r_e/R_ref  +  w_t * t_e/T_ref  +  w_E * E_e/E_ref  +  w_d * d_e/D_ref  -  w_sci * science_e/S_ref
```

Mode	w_s (safety)	w_t (time)	w_E (energy)	w_d (distance)	w_sci	Behaviour
Safest	1.00	0.02	0.05	0.02	0	Avoids slope, dust, radiation and long shelter distance even if much longer. Strictest hard limits.
Balanced	0.50	0.20	0.20	0.10	0	Good default for mission planning.
Fastest	0.15	0.65	0.10	0.10	0	Direct, still respects every hard constraint. Mirrors your ELYSIUM 'Fastest' idea (0.85 distance / 0.15 slope).
Shortest	0.05	0	0	0.95	0	Minimum distance subject to hard constraints only.
Low-energy	0.30	0.10	0.55	0.05	0	Longest battery life; avoids climbs.
Science	0.40	0.10	0.15	0.05	0.30	Takes detours to science targets inside the risk budget.
Custom	user	user	user	user	user	Slider UI; weights must sum to 1.

Important: in **every** mode the hard constraints and the shelter-reachability prune stay on. Modes only change preferences among already-safe routes.

5.1 Route statistics and scores (for your Mission Dashboard)

```
Total risk R           = sum r_e                        Safety score  = 100 * exp(-R)
Distance, time, gain/loss, max/mean slope, max/mean roughness, hazard exposure = sum( d_e * h_e )
Radiation dose         = sum dose_e                     O2 use, battery use from Section 3.7
Min shelter distance   = max over route of T_shelter    Worst-edge risk = max r_e   (bottleneck)
Efficiency             = straight_time / route_time     Science score = sum science_e
Overall mission score  = w1*Safety + w2*Efficiency + w3*Science + w4*ResourceMargin     (weights sum to 1)
```

5.2 Custom-route audit (user draws a path)

```
for every 100 m segment of the polyline:
      evaluate the same edge() function  ->  r_e, slope, slip, dose, O2, flags
      flag RED if  hard constraint broken,  or  r_e > 3 * median r_e,  or  slope > 15 deg
Viability % = 100 * exp(-R_total)  *  (0 if any hard constraint broken)
Output: segment table, red segments on the map, total risk, comparison with the planned safest route
```

6. Putting it into the Sanket Grid, build order and validation

6.1 New columns for your central grid table (one authoritative value per cell)

Group	Columns to add
Geometry	z, slope_1m, slope_5m, slope_20m, aspect, curv_profile, curv_plan, rough_1m/5m/25m, step_height, dist_cliff, clearance, svf
Surface	terrain_class, s0, phi, kc, kphi, n, rock_cfa, boulder_max_h, dune_flag, dune_depth, thermal_inertia, albedo, void_flag, streak_flag, label_conf
Atmosphere (time series)	pressure, rho, wind_mean, wind_gust, wind_dir, tau, visibility, dust_storm_flag
Thermal / power	T_min, T_max, T(t), sun_incidence(t), irradiance(t), shadow_flag
Radiation	D0, D_local, svf_factor, spe_shelter_time, shelter_flag
Comms	comm_block, los_hab, los_relay, loc_quality
Uncertainty	sigma_z, sigma_slope, sigma_class, data_age, nodata_flag
Derived routing	hazard_hard (0/1), r_dist_rate, r_time_rate, h_* per factor, T_shelter, o2_rate, E_rate, final_weighted_cost per mode

Compute the hazard fields once and store them; the Route Engine, SOALM assistant, Unity and the Website then all read the same numbers (your 'one authoritative value' rule).

6.2 Suggested build order

- **Step 1 (day 1):** slope, cross-slope, roughness, rocks, dunes -> hard mask + slope/roughness risk. A* on this gives a basic safe route.
- **Step 2:** add shelter field and the walk-back prune; add radiation shielding via sky-view factor.
- **Step 3:** add dust, wind, temperature, comms as per-hour terms (even as constant layers at first).
- **Step 4:** add oxygen / battery / time budgets (Lagrangian wrapper) and the mode presets.
- **Step 5:** add uncertainty, alternatives, rerouting, and the custom-route audit.
- **Step 6:** connect SOALM: natural language -> parameters (start, goal, max slope, objective) -> plan() -> explanation of the result.

6.3 Validation (this is what makes it credible to judges)

- **Constraint check:** independently re-test every cell of the output route against all hard limits.
- **Replay real drives:** compare predicted slope / slip / energy for Perseverance traverse segments in Jezero with the logged values; tune s0, phi and the lambdas.
- **Sensitivity analysis:** perturb each weight and threshold by +/-20 % and see how the route changes; report which factors dominate.
- **Monte Carlo:** sample sigma_z, label errors and tau; report the 90th-percentile risk and how often the route stays valid.
- **Baselines:** show the safest route vs shortest route on the same map, with the risk, dose and slope numbers side by side.
- **Expert weights:** for the lambda / mu values, use pairwise comparison (AHP) with a domain expert or published mission rules, and document it.

6.4 Common mistakes to avoid

- Adding a per-cell cost once per cell instead of integrating over distance and time (creates diagonal bias).
- Letting a weighted sum trade a hard limit against distance. Hard limits must be removed from the graph, not penalised.
- Normalising each route's values by that route's own min / max; use fixed reference values.
- Treating missing data as safe. Unknown is blocked or maximally costly.
- A zero-cost corridor with no tie-breaker; always keep the small eps * distance term.
- Using Earth-calibrated models (Minetti, slip curves) without noting the gravity and soil differences.

Appendix: reference implementation (Python, numpy)

Layers are 2-D numpy arrays on the grid. This was run on a synthetic 60 x 60 Mars-like grid with a hill between start and goal; it returned a valid 71-cell route. The input keys are: dem, rough, rock, dune, tau, gust, temp, rad_day, comm_block, clear_m, s0, phi, blocked. Use `plan()` for the full route with limits, and `audit()` for custom polylines.

```
"""MarsWalk safest-route engine - reference implementation.

Cost = risk (-ln survival, additive) + Lagrange multipliers * resources.
Layers are 2-D numpy arrays on the Sanket Grid (row, col).
"""
import heapq
import math
import numpy as np

G_MARS = 3.71


def ramp(x, a, b):
    """0 at/below a (safe), 1 at/above b (limit)."""
    return float(np.clip((x - a) / (b - a), 0.0, 1.0))


DEFAULT = dict(
    cell=10.0,           # metres per cell
    v0=1.0,              # nominal speed on flat ground, m/s
    max_slope=15.0, slope_safe=8.0,      # deg, along-heading
    max_cross=18.0, cross_safe=8.0,      # deg, side slope (tip-over)
    max_slip=0.6, q=2.0,                 # q>1: mild values cheap, bad costly
    lam=dict(slope=1e-3, cross=1e-3, slip=1.2e-3, rough=8e-4,
             rock=8e-4, dune=1e-3, clear=5e-4),   # risk units per metre
    mu=dict(dust=0.4, wind=0.3, cold=0.2, comm=0.2, spe=0.5),  # per hour
    k_rad=0.05,          # risk units per mSv (linear no-threshold)
    d0=30.0,             # clearance decay distance, m
    t_warn_h=1.0,        # SPE warning lead time, h
    beta=0.0,            # risk units per hour of EVA (time pressure)
    eps=1e-5,            # tiny per-metre cost so routes stay short
    met=4.0, body_kg=80.0,               # metabolic load for O2 model
)


def build_fields(L, P):
    """Per-cell risk rates from raw layers. L keys: dem, rough, rock, dune,
    tau, gust, temp, rad_day, comm_block, clear_m, s0, phi, blocked."""
    q = P['q']
    lam, mu = P['lam'], P['mu']
    H, W = L['dem'].shape
    F = dict(L)
    gr, gc = np.gradient(L['dem'], P['cell'])
    F['gr'], F['gc'] = gr, gc
    lam_c = np.zeros((H, W))
    mu_c = np.zeros((H, W))
    for i in range(H):
        for j in range(W):
            lam_c[i, j] = (
                lam['rough'] * ramp(L['rough'][i, j], 0.1, 0.3) ** q
                + lam['rock'] * ramp(L['rock'][i, j], 5, 20) ** q
                + lam['dune'] * L['dune'][i, j] ** q
                + lam['clear'] * math.exp(-L['clear_m'][i, j] / P['d0']))
            mu_c[i, j] = (
                mu['dust'] * ramp(L['tau'][i, j], 1.0, 4.0) ** q
                + mu['wind'] * ramp(L['gust'][i, j], 15, 30) ** q
                + mu['cold'] * ramp(200.0 - L['temp'][i, j], 0, 40) ** q
                + mu['comm'] * L['comm_block'][i, j])
    F['lam_c'], F['mu_c'] = lam_c, mu_c
    return F


def edge(P, F, a, b, rad_key='rad_day'):
    """Return dict(risk, t_h, dose, o2, energy, d) or None if forbidden."""
    (i, j), (k, l) = a, b
    if F['blocked'][k, l]:
        return None
    dc = math.hypot(k - i, l - j)
    d = dc * P['cell']
    ux, uy = (l - j) / dc, (k - i) / dc          # heading (col,row)
    tan_al = (F['dem'][k, l] - F['dem'][i, j]) / d
    gx = 0.5 * (F['gc'][i, j] + F['gc'][k, l])
    gy = 0.5 * (F['gr'][i, j] + F['gr'][k, l])
    tan_cr = abs(-gx * uy + gy * ux)
    th_al = math.degrees(math.atan(tan_al))
    th_cr = math.degrees(math.atan(tan_cr))
    if abs(th_al) > P['max_slope'] or th_cr > P['max_cross']:
        return None
    s0 = 0.5 * (F['s0'][i, j] + F['s0'][k, l])
    phi = 0.5 * (F['phi'][i, j] + F['phi'][k, l])
```

```
    slip = s0 + (1 - s0) * min(1.0, max(tan_al, 0) /
                               math.tan(math.radians(phi))) ** 2
    if slip > P['max_slip']:
        return None
    q = P['q']
    lam_m = 0.5 * (F['lam_c'][i, j] + F['lam_c'][k, l])
    risk = d * (lam_m
                + P['lam']['slope'] * ramp(abs(th_al), P['slope_safe'],
                                           P['max_slope']) ** q
                + P['lam']['cross'] * ramp(th_cr, P['cross_safe'],
                                           P['max_cross']) ** q
                + P['lam']['slip'] * ramp(slip, 0.2, P['max_slip']) ** q)
    # speed: Tobler slope factor (capped at flat speed) x traction
    tob = math.exp(-3.5 * (abs(tan_al + 0.05) - 0.05))
    v = max(P['v0'] * (1 - slip) * min(1.0, tob), 0.05)
    t_h = d / v / 3600.0
    mu_m = 0.5 * (F['mu_c'][i, j] + F['mu_c'][k, l])
    rad = 0.5 * (F[rad_key][i, j] + F[rad_key][k, l])        # mSv/day
    dose = rad / 24.0 * t_h
    risk += mu_m * t_h + P['k_rad'] * dose
    o2 = P['met'] * 3.5 * P['body_kg'] * 60 * t_h / 1000 * 1.429 / 1000
    return dict(risk=risk, t_h=t_h, dose=dose, o2=o2, d=d)


NB8 = [(-1, 0), (1, 0), (0, -1), (0, 1), (-1, -1), (-1, 1), (1, -1), (1, 1)]


def neighbours(F, n, eight=True):
    H, W = F['dem'].shape
    for di, dj in (NB8 if eight else NB8[:4]):
        m = (n[0] + di, n[1] + dj)
        if not (0 <= m[0] < H and 0 <= m[1] < W):
            continue
        if di and dj and (F['blocked'][n[0] + di, n[1]] or
                          F['blocked'][n[0], n[1] + dj]):
            continue                                  # no corner cutting
        yield m


def time_to_shelter(P, F, shelters):
    """Backward Dijkstra: hours from every cell to nearest shelter."""
    H, W = F['dem'].shape
    dist = np.full((H, W), np.inf)
    pq = []
    for s in shelters:
        dist[s] = 0.0
        heapq.heappush(pq, (0.0, s))
    while pq:
        t, n = heapq.heappop(pq)
        if t > dist[n]:
            continue
        for m in neighbours(F, n):
            e = edge(P, F, m, n)
            if e and t + e['t_h'] < dist[m]:
                dist[m] = t + e['t_h']
                heapq.heappush(pq, (dist[m], m))
    return dist


def add_spe_risk(P, F, shelters):
    """Penalise cells whose shelter is farther than the SPE warning time."""
    F['t_shel_h'] = time_to_shelter(P, F, shelters)
    H, W = F['dem'].shape
    for i in range(H):
        for j in range(W):
            F['mu_c'][i, j] += P['mu']['spe'] * ramp(
                F['t_shel_h'][i, j], P['t_warn_h'], 2 * P['t_warn_h'])
    return F


def astar(P, F, start, goal, mult=None, budget=None, margin=1.3):
    """Least-cost path. mult = Lagrange multipliers on resources.
    budget = dict(t_h=..) enables the return-to-shelter prune."""
    mult = mult or {}
    cmin = P['eps'] + (P['beta'] + mult.get('t_h', 0)) / (P['v0'] * 3600)
    sq = math.sqrt(2)

    def h(n):
        dx, dy = abs(n[0] - goal[0]), abs(n[1] - goal[1])
        return cmin * P['cell'] * (dx + dy + (sq - 2) * min(dx, dy))

    zero = dict(risk=0, t_h=0, dose=0, o2=0, d=0)
    best = {start: 0.0}
    stats = {start: zero}
    parent = {}
    pq = [(h(start), 0.0, start)]
    while pq:
        f, g, n = heapq.heappop(pq)
        if n == goal:
            path = [n]
```

```
            while n in parent:
                n = parent[n]
                path.append(n)
            return path[::-1], stats[goal]
        if g > best[n]:
            continue
        for m in neighbours(F, n):
            e = edge(P, F, n, m)
            if e is None:
                continue
            c = (e['risk'] + P['eps'] * e['d'] + P['beta'] * e['t_h']
                 + sum(mult.get(k, 0) * e[k] for k in ('t_h', 'dose', 'o2')))
            st = {k: stats[n][k] + e[k] for k in zero}
            if budget and 't_shel_h' in F and \
                    st['t_h'] + margin * F['t_shel_h'][m] > budget['t_h']:
                continue                        # cannot get home from there
            if g + c < best.get(m, 1e18):
                best[m], stats[m], parent[m] = g + c, st, n
                heapq.heappush(pq, (g + c + h(m), g + c, m))
    return None, None


def plan(P, F, start, goal, limits=None, budget=None):
    """Lagrangian wrapper: raise multipliers until route meets limits.
    limits e.g. dict(t_h=6, dose=0.5, o2=0.6)."""
    limits = limits or {}
    mult = {k: 0.0 for k in limits}
    best = None
    for _ in range(25):
        path, st = astar(P, F, start, goal, mult, budget)
        if path is None:
            return best
        over = [k for k in limits if st[k] > limits[k]]
        if not over:
            return path, st, mult
        best = best or (path, st, mult)
        for k in over:
            mult[k] = mult[k] * 2 if mult[k] else 0.05
    return best


def audit(P, F, path):
    """Segment-by-segment risk of any (custom) polyline of grid cells."""
    out = []
    for a, b in zip(path[:-1], path[1:]):
        e = edge(P, F, a, b)
        out.append(None if e is None else e['risk'])
    return out


def survival(stats):
    return math.exp(-stats['risk'])
```